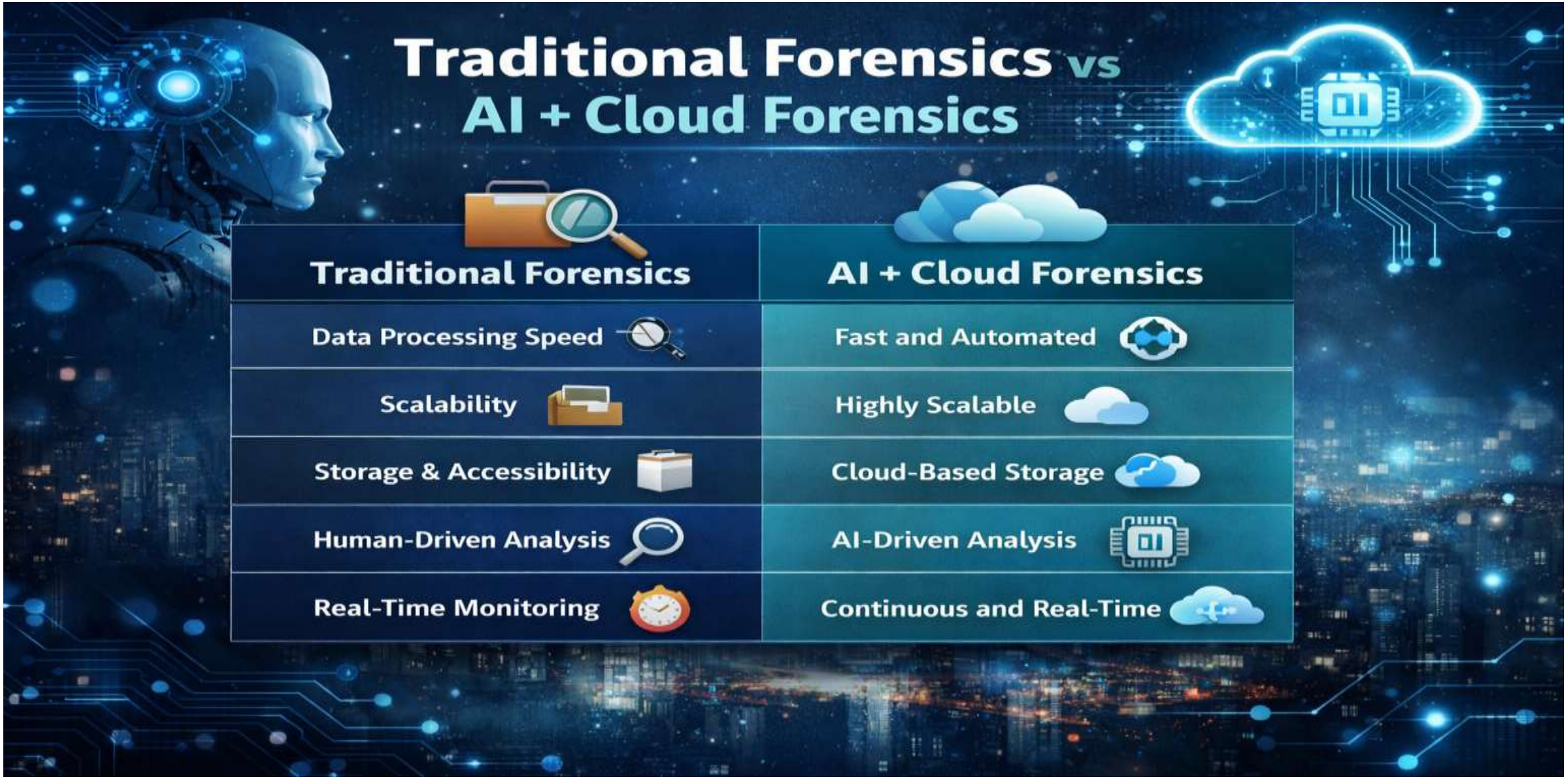
TITLE:AI-BASED CYBER FORENSIC INVESTIGATION ASSISTANT USING CLOUD COMPUTING

MEMBER: Syeda Hafsa Fathima
INSTITUTE: SR UNIVERSITY
DATE:07/04/2026
HALL TICKET NO:2303A52381
BATCH:39

INTRODUCTION:

The field of digital forensics is becoming increasingly critical due to the rising frequency and complexity of cybercrimes. Traditional manual forensic methods are often slow, inefficient, and prone to human error. To address these challenges, this project introduces an AI-based cyber forensic assistant hosted on cloud infrastructure. By combining artificial intelligence with scalable cloud computing, the system can automatically analyze large volumes of digital evidence, triage suspicious items, and generate structured reports, allowing investigators to focus on critical tasks while improving accuracy and speed.



This fig shows difference between traditional and ai linked with cloud forensics

Diagram showing “Evidence → Cloud → AI Assistant → Triage Report → Investigator

PROBLEM STATEMENT:

Digital forensic investigations face several challenges, including large-scale data handling, slow manual analysis, and difficulty in prioritizing critical evidence. During cyber incidents, such as data breaches or ransomware attacks, timely identification of high-risk evidence is crucial. Manual analysis often delays investigation outcomes and may overlook important anomalies. This project solves these problems by implementing an AI-powered assistant hosted on the cloud, capable of automated triage, anomaly detection, and generating actionable reports, thus bridging the gap between manual limitations and the need for rapid, accurate forensic analysis

Feature / Aspect	Traditional Forensics	AI + Cloud Forensics
Data Processing Speed	Slow, manual analysis	Fast, automated with AI algorithms
Data Volume Handling	Limited; struggles with large datasets	Handles massive datasets in the cloud easily
Accuracy	Depends on human expertise; prone to error	High accuracy due to AI pattern recognition
Accessibility	Physical access to devices needed	Remote access via cloud platforms
Cost	High (tools + manpower)	Reduced long-term cost due to automation
Scalability	Difficult to scale	Highly scalable via cloud infrastructure
Investigation Time	Longer due to manual steps	Significantly shorter with AI automation
Evidence Preservation	Manual chain-of-custody management	Automated logs & tracking in cloud
Learning & Adaptation	Static procedures	AI can learn from new cases and improve

Comparison table: Traditional Forensics vs AI + Cloud Forensics.

OBJECTIVES:

- The project aims to achieve the following objectives:
- **Automate Evidence Collection:** Streamline the process of gathering logs, emails, network packets, and other digital artifacts.
- **AI-Assisted Analysis:** Use machine learning models to detect anomalies, flag suspicious activities, and prioritize evidence.
- **Cloud-Based Scalability:** Host the system on cloud infrastructure to allow remote access, high-volume processing, and secure storage.
- **Report Generation:** Provide structured, ready-to-use forensic triage reports for investigators and law enforcement teams.
  These objectives ensure the system is efficient, accurate, and scalable, directly addressing the shortcomings of traditional forensic methods.

In this figure ,The system uses AI and cloud computing to automate analysis, generate reports, and improve the efficiency of cyber forensic investigations.

STAKEHOLDERS:

The primary stakeholders include digital forensic investigators, cybersecurity analysts, and law enforcement agencies who directly benefit from faster, AI-assisted investigations.

Secondary stakeholders include corporate IT teams, organizations needing forensic support, and cloud service providers ensuring secure, scalable infrastructure. By addressing the needs of these stakeholders, the system not only aids in investigation efficiency but also strengthens cybersecurity operations across sectors.

TOOLS, PROGRAMMING LANGUAGES,PLATFROMS:

The project utilizes a combination of modern programming languages, AI libraries, and cloud platforms. Python is used for AI model development and data preprocessing, while JavaScript enables an interactive user interface. Key libraries include Pandas, NumPy, and scikit-learn for data handling and machine learning. Cloud computing is implemented using AWS, with S3 for secure storage and EC2 for scalable processing. MongoDB stores structured evidence data, and additional forensic tools like FTK Imager and Wireshark are used to extract and preprocess raw digital evidence. This combination ensures a robust, efficient, and scalable forensic solution.

PROMPT:CYBER FORENSIC AI - A SIMPLE FLASK APP FOR ANALYZING TEXT AND FILES FOR POTENTIAL CYBER THREATS.

PYTHON CODE:

```python
# Cyber Forensic AI - A simple Flask app for analyzing text and files for potential cyber threats.
from flask import Flask, render_template, request, jsonify
import hashlib
import datetime
import os

app = Flask(__name__)

# create cloud storage folder
if not os.path.exists("cloud_storage"):
    os.makedirs("cloud_storage")

def analyze_file(data):
    if not data:
        data = ""

    hash_value = hashlib.md5(data.encode()).hexdigest()

    threat = "Low"
    analysis = "No suspicious activity detected."

    keywords = ["attack", "malware", "virus", "hack"]
```

```
PROBLEMS   OUTPUT   DEBUG CONSOLE   TERMINAL   PORTS   SQL LANGUAGE SERVER
PS C:\Users\syeda\OneDrive\Desktop\VS Code Cp> & C:\Users\syeda\AppData\Local\Python\pythoncore-3.14-64\python.exe "c:/Users/syeda/OneDrive/Desktop/VS Code Cp/cyber-forensic-ai/app.py"
 * Serving Flask app 'app'
 * Debug mode: on
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
 * Running on http://127.0.0.1:5000
Press CTRL+C to quit
 * Restarting with stat
```

```python
    for word in keywords:
        if word in data.lower():
            threat = "High"
            analysis = "Suspicious activity detected!"

    timestamp = str(datetime.datetime.now())

    result = {
        "hash": hash_value,
        "threat_level": threat,
        "analysis": analysis,
        "timestamp": timestamp
    }

    # Save report to "cloud"
    filename = f"cloud_storage/report_{hash_value}.txt"
    with open(filename, "w", encoding="utf-8") as f:
        f.write(str(result))
```

```python
def analyze_file(data):
    return result

@app.route('/')
def home():
    return render_template("index.html")

@app.route('/analyze', methods=['POST'])
def analyze():
    data = request.form.get("data")

    if 'file' in request.files:
        file = request.files['file']
        if file.filename != "":
            data = file.read().decode('utf-8')

    result = analyze_file(data)
    return jsonify(result)

if __name__ == '__main__':
    app.run(debug=True)
```

Generate a simple HTML frontend for a cyber forensic investigation assistant that allows users to input text or upload a file, submit it for analysis, and display the threat

Run Terminal Help VS Code Cp

ent 1.py | ai project.py | app.py | report_03b3361fc58d5e3d22bf93778e60217b.txt

cyber-forensic-ai > templates > index.html > html

```html
<!DOCTYPE html>
<html>
<head>
    <title>Cyber Forensic AI Assistant</title>
    <link rel="stylesheet" href="/static/style.css">
</head>
<body>

<h1>🔍 AI Cyber Forensic Investigation Tool</h1>

<!-- ☁️ Cloud feature -->
<p style="color: lightgreen;">☁️ Cloud Storage Enabled</p>

<!-- Input area -->
<textarea id="inputData" placeholder="Paste logs..."></textarea>

<br><br>

<!-- File upload -->
<input type="file" id="fileInput">

<br><br>

<!-- Button -->
<button onclick="analyzeData()">Analyze</button>

<h2>Result:</h2>

<!-- Output display -->
<p><b>Hash:</b> <span id="hash"></span></p>
<p><b>Threat Level:</b> <span id="threat"></span></p>
<p><b>Analysis:</b> <span id="analysis"></span></p>
<p><b>Timestamp:</b> <span id="time"></span></p>

<!-- JS link -->
<script src="/static/script.js"></script>

</body>
</html>
```

Generate a javascript (script.js) file that handles form submission for a cyber forensic investigation assistant, sends user input or file data to a flask backend using a post request, and displays the response including threat level, hash value, and timestamp dynamically on the webpage.”

cyber-forensic-ai > static > JS script.js > analyzeData

```javascript
function analyzeData() {
    let data = document.getElementById("inputData").value;
    let fileInput = document.getElementById("fileInput").files[0];

    let formData = new FormData();
    formData.append("data", data);

    if (fileInput) {
        formData.append("file", fileInput);
    }

    fetch('/analyze', {
        method: 'POST',
        body: formData
    })
    .then(res => res.json())
    .then(result => {
        document.getElementById("hash").innerText = result.hash;
        document.getElementById("threat").innerText = result.threat_level;
        document.getElementById("analysis").innerText = result.analysis;
        document.getElementById("time").innerText = result.timestamp;
    });
}
```

Style.css:

cyber-forensic-ai > static > # style.css > button

```
body {
    font-family: Arial;
    text-align: center;
    background-color: #0f172a;
    color: white;
}

textarea {
    width: 60%;
    height: 150px;
    padding: 10px;
}

button {
    padding: 10px 20px;
    background-color: green;
    color: white;
    border: none;
}
```

OUTPUT:

Cyber Forensic AI Assistant
127.0.0.1:5000
Chat
AI Cyber Forensic Investigation Tool
Cloud Storage Enabled
this is a malware attack
Choose File
No file chosen
Analyze
Result:
Hash: 03b3361fc58d5e3d22bf93778e60217b
Threat Level: High
Analysis: Suspicious activity detected!
Timestamp: 2026-04-06 22:36:10.418704

o Run Terminal Help VS Code Cp 1

mnent 1.py | ai project.py | app.py | report_03b3361fc58d5e3d22bf93778e60217b.txt | index.html | style.css | script.js

cloud_storage > report_03b3361fc58d5e3d22bf93778e60217b.txt


```
1  {'hash': '03b3361fc58d5e3d22bf93778e60217b', 'threat_level': 'High', 'analysis': 'Suspicious activity detected!', 'timestamp': '2026
```

Website URL:

- http://127.0.0.1:5000/

FLOWCHART OF THE WORK FLOW:

- **User Input / File Upload**
- ↓
- **Frontend (HTML/JS)**
- ↓
- **Flask Backend (app.py)**
- ↓
- **Data Analysis + Hashing**
- ↓
- **Threat Detection**
- ↓
- **Store in Cloud (cloud_storage)**
- ↓
- **Display Result**

REFERENCES:

- Python Software Foundation, *Python Documentation*
- Pallets Projects, *Flask Documentation*
- Mozilla Developer Network (MDN), *HTML, CSS, JavaScript Docs*
- W3Schools, *Web Development Tutorials*
- OpenAI, *ChatGPT (AI assistance for code generation)*